Adam: Momentum and RMSProp Together

Contents

1. Overview 2
2. Prerequisites 3
3. The story so far 4
4. The update rule 5
5. Bias correction 7
6. Adam on the students data 8
7. Adam on real data: MNIST 10
8. Which optimizer to use 11
9. Adam in Keras 12
10. Summary 13
11. Sources 13
12. Key terms 14

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Optimizers in deep learning; Exponentially weighted moving average (EWMA); SGD with momentum; RMSProp.
- **Leads to:** Learning-rate warm-up schedule.
- **Compare with:** RMSProp.

1. Overview

Key point: Adam keeps two running averages (EWMAs) per parameter: of the gradient (momentum's idea) and of the squared gradient (RMSProp's idea). It corrects both for starting at 0, then steps by the first divided by the root of the second. It is the most widely used optimizer and the usual starting point.

Adam (G-169), short for *adaptive moment estimation* (Kingma and Ba 2015), is the last **optimizer** (G-1401) of this series and the most used. Whether we train a plain network, a **convolutional network** (G-484; a network for images) or a **recurrent network** (G-1647; a network for sequences such as text), Adam is usually the first choice.

Adam borrows from the optimizers before it. **Momentum** (G-1258) and **NAG** (G-1315) are built on one idea, **speed from past gradients**; **AdaGrad** (G-168) and **RMSProp** (G-1697) on another, **a learning rate that adapts to each parameter**. Adam merges the two.

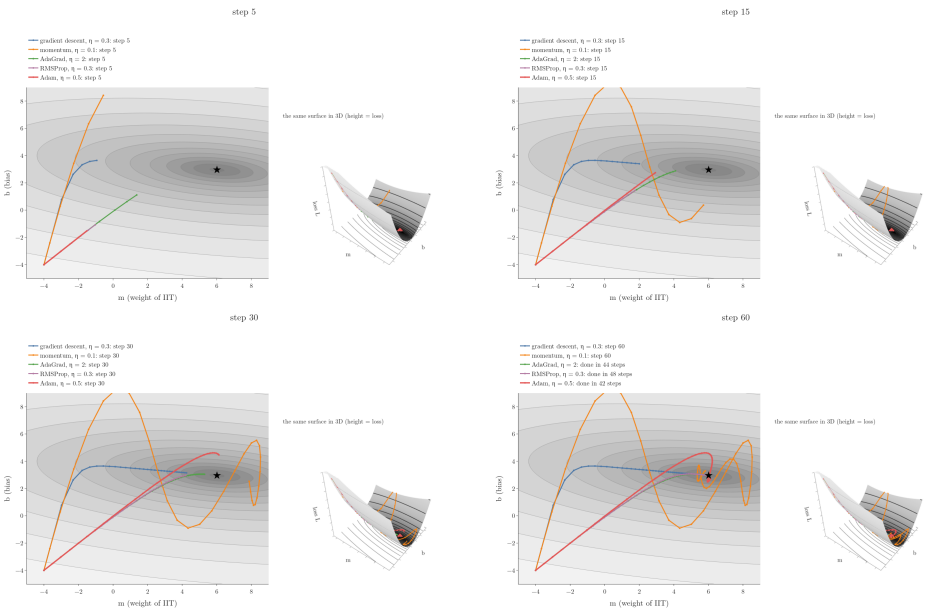


Figure 1: Five optimizers on the elongated bowl of a sparse feature, each with a learning rate that works for it. Adam (red) heads towards the minimum like the adaptive methods and curls in like momentum

Figure 1 is a **contour map** (the surface seen from above; each line joins points of equal loss; how to read a contour map), with the same surface in 3D at its right (height = the loss; the lines on it are the lines of the map). The surface is the loss of the students data, a bowl stretched along m (built, and tilted from the side to the top view, in a sparse feature stretches the loss):

$$L(m, b) = \frac{1}{100} \sum_{i=1}^{100} (y_i - (m\, x_i + b))^2$$

At the start of the paths:

$$L(-4, -4) = 72.63$$

Here x_i is the IIT value (0 or 1) of student i and y_i their package.

Lines close together mean steep (across the bowl, along b), lines far apart mean flat (along m), and the star is the lowest point, $L(6.01, 2.96) = 0.23$.

Figure 1 shows both ideas at work on the students data.

2. Prerequisites

- Momentum's velocity, a running average of past gradients.
- RMSProp's division by the root of an EWMA of squared gradients.

- Why an average started at 0 is too small at first.

3. The story so far

> **Key point:** Batch GD is steady but slow; momentum is fast but overshoots; NAG damps the overshoot; AdaGrad handles sparse features but its **learning rate** (G-1068) dies; RMSProp fixes that.

Optimizer	What it adds	Its problem
Batch gradient descent	the basic step	slow
Momentum (G-1258)	speed from past gradients	overshoots and oscillates
NAG	look ahead, damp the oscillations	can stay in small dips
AdaGrad (G-168)	a learning rate per parameter (sparse features)	the learning rate only shrinks
RMSProp (G-1697)	forget old gradients	no correction for starting at 0

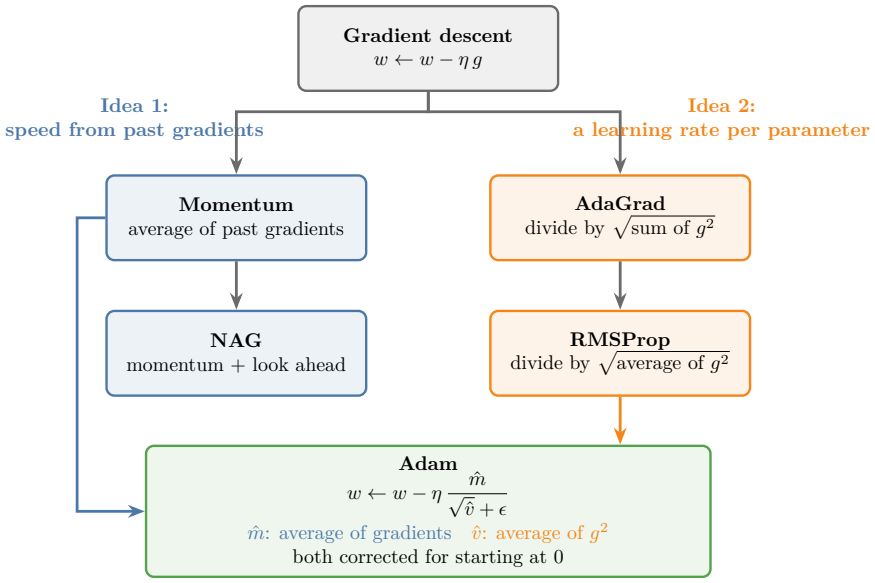


Figure 2: The two lines of optimizers. Left (blue): speed from past gradients. Right (orange): a learning rate per parameter. Adam takes the average of gradients from the left line and the average of squared gradients from the right line

Figure 2 draws the table as a family tree. Follow the two coloured arrows into Adam: the blue one brings momentum's average of past gradients, the orange one brings RMSProp's average of squared gradients.

The two lines of improvement are independent, so it makes sense to combine them. Adam is perhaps best seen as a combination of RMSProp and momentum, with a few important differences (Goodfellow et al. 2016, §8.5.3).

4. The update rule

> **Key point:** Three lines per step: update the two averages, correct both for starting at 0, then move by the corrected average gradient divided by the root of the corrected average squared gradient.

1. **In words:**
   - keep an EWMA (**exponentially weighted moving average**, G-735) of the gradient, m_t, as **momentum** (G-1258) does;
   - keep an EWMA of the squared gradient, v_t, as **RMSProp** (G-1697) does;
   - correct both for having started at 0;
   - step in the direction of the corrected average gradient, divided by the root of the corrected average squared gradient.

2. **Formula:**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, \nabla L(w_t)$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, (\nabla L(w_t))^2$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t$$

   with $m_0 = v_0 = 0$, where t counts the updates: 1 for the first mini-batch, 2 for the second, and so on. The suggested defaults are $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$ (Kingma and Ba 2015, algorithm 1).

   β_2 is much closer to 1 than β_1, so v_t averages over many more past gradients than m_t does. A reliable estimate of the squared gradient needs that long average, above all when most gradients are zero (sparse gradients) (Kingma and Ba 2015, §3, §6.4).

3. **Example:** one weight with gradients 2 and then 1, using the defaults.

   Step $t = 1$, gradient 2:

$$m_1 = 0.1 \times 2 = 0.2$$
$$v_1 = 0.001 \times 2^2 = 0.004$$
$$\hat{m}_1 = 0.2/0.1 = 2$$
$$\hat{v}_1 = 0.004/0.001 = 4$$
$$\text{step} = 0.001 \times 2/\sqrt{4}$$

$$= 0.001$$

Step $t = 2$, gradient 1:

$$m_2 = 0.9 \times 0.2 + 0.1 \times 1$$
$$= 0.28$$
$$v_2 = 0.999 \times 0.004 + 0.001 \times 1^2$$
$$= 0.004996$$
$$\hat{m}_2 = 0.28/(1 - 0.9^2) = 0.28/0.19$$
$$= 1.474$$
$$\hat{v}_2 = 0.004996/(1 - 0.999^2)$$
$$= 0.004996/0.001999 = 2.499$$
$$\text{step} = 0.001 \times 1.474/\sqrt{2.499}$$
$$= 0.000932$$

(Notebook.)

The parts come from the earlier **optimizers** (G-1401):

- m_t is momentum's velocity, written as a true EWMA. In **Adam** (G-169) it is an estimate of the **first moment** (G-780; the mean) of the gradient (Goodfellow et al. 2016, §8.5.3).
- $\sqrt{v_t}$ **in the denominator** is RMSProp's. v_t estimates the **second moment** (G-1759; the mean of the square) of the gradient, which gives Adam its name: adaptive **moment** estimation.
- $\hat{m}_t$ **and** $\hat{v}_t$ are new: the **bias correction** (G-285).

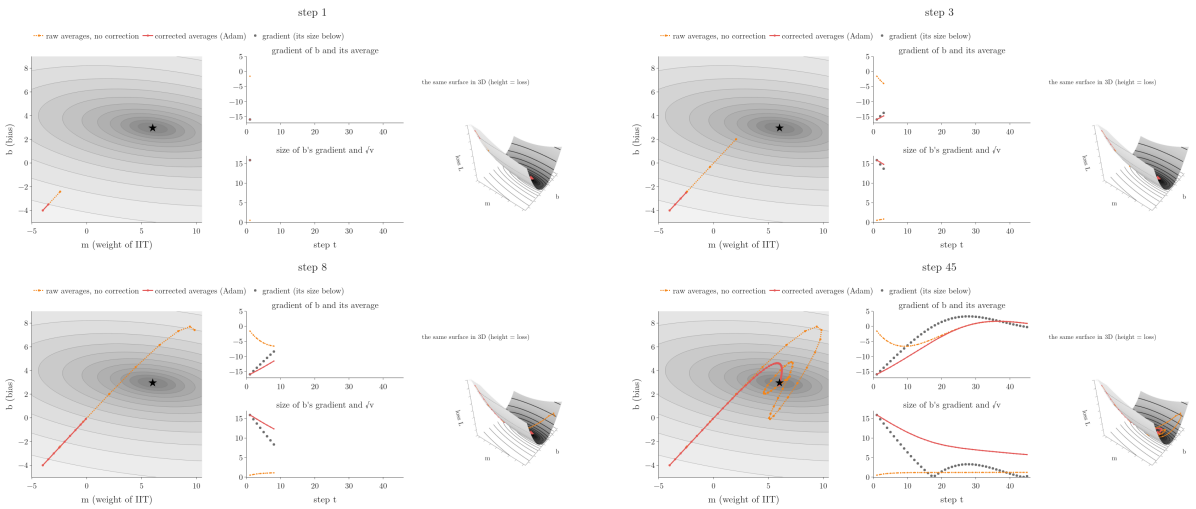


Figure 3: Adam, $\eta = 0.5$, on the students data, step by step. Left: the path with corrected averages (red) and with the raw ones (dotted orange). Right, for the bias b of the red run: the gradient (grey dots) with m raw and corrected (top), and the gradient's size with $\sqrt{v}$ raw and corrected (bottom)

Figure 3 runs the rule on the students data of Figure 1. Watch the right panels: the raw averages start near 0 (at step 1, $\sqrt{v}$ is 0.50 while the gradient's size is 15.9), and the corrected ones start on the gradient. With the raw averages the first step is 2.24 long instead of 0.71, and the path overshoots to $b = 7.7$ before it comes back; section 5 explains why.

5. Bias correction

> **Key point:** An EWMA started at 0 is too small at first: after t steps its weights add up to only $1 - \beta^t$ (0.1 after one step with $\beta = 0.9$). Dividing by $1 - \beta^t$ removes this start-up bias. With $\beta_2 = 0.999$ the bias would last for thousands of steps.

Both averages start at $m_0 = v_0 = 0$, so in the first steps they are pulled towards 0, exactly like the zero start of an EWMA.

How big the bias is. Unrolled, the EWMA after t steps is a weighted sum of the gradients $g_1, \ldots, g_t$, with weight $(1 - \beta)\beta^{t-i}$ on gradient g_i:

$$(1 - \beta) \sum_{i=1}^{t} \beta^{t-i} g_i$$

If every gradient had the same value g, this would be

$$(1 - \beta) \left(1 + \beta + \cdots + \beta^{t-1}\right) g$$

$$= \left(1 - \beta^t\right) g$$

because the geometric sum $1 + \beta + \cdots + \beta^{t-1}$ equals $(1 - \beta^t)/(1 - \beta)$. Check with $\beta = 0.9$ and $t = 3$:

$$1 + 0.9 + 0.81 = 2.71$$
$$\frac{1 - 0.9^3}{1 - 0.9} = \frac{1 - 0.729}{0.1} = 2.71$$

The average is too small by exactly the factor $1 - \beta^t$, so dividing by it gives back g (Kingma and Ba 2015, §3).

	$t = 1$	$t = 10$	$t = 100$	$t = 1000$
$1 - 0.9^t$	0.1	0.651	1.000	1.000
$1 - 0.999^t$	0.001	0.010	0.095	0.632

With $\beta_1 = 0.9$ the factor reaches 1 within a few dozen steps; with $\beta_2 = 0.999$ it is still 0.63 after 1,000 steps (Notebook). The larger β is, the longer the bias lasts (Kingma and Ba 2015, §2).

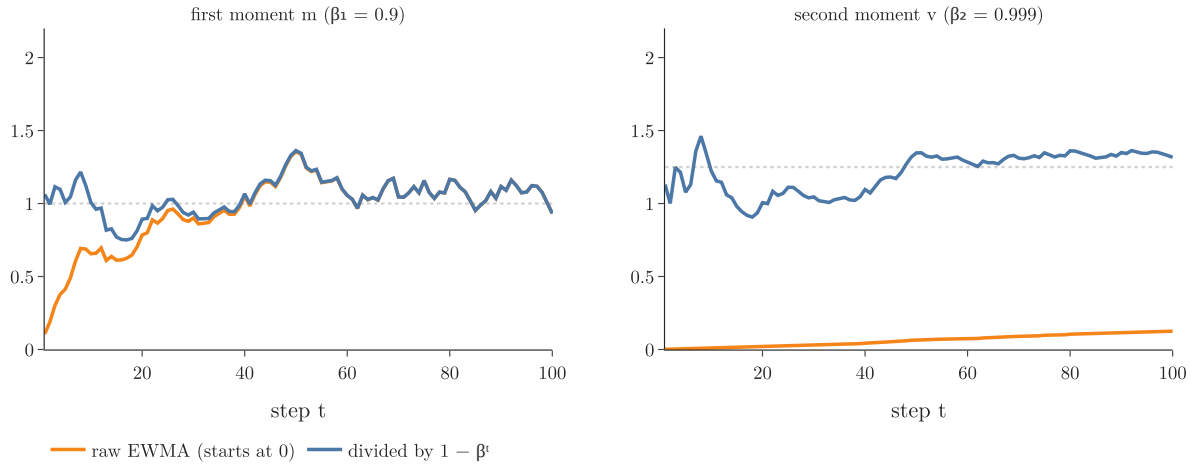


Figure 4: EWMAs of a noisy gradient with mean 1 (true second moment 1.25), started at 0. The raw averages (orange) start near 0 and climb slowly, very slowly for $\beta_2 = 0.999$; the corrected ones (blue) are on target from the first step

Figure 4 shows both averages on a noisy gradient. At step 10, the raw v is 0.012 against a true value of 1.25; corrected, it is 1.22 (Notebook).

Without the correction, early steps would be badly scaled: at $t = 1$ in the example above, the raw ratio is

$$m_1/\sqrt{v_1} = 0.2/\sqrt{0.004}$$

$$= 3.16$$

instead of the corrected 1, a first step more than three times too large. Kingma and Ba (2015, §3) point out that leaving the correction out leads to much larger initial steps. **RMSProp** (G-1697) keeps an uncorrected second-moment estimate, which may be strongly biased early in training (Goodfellow et al. 2016, §8.5.3).

6. Adam on the students data

> **Key point:** Adam heads for the minimum like an adaptive method, curls in like **momentum** (G-1258), and then settles, while RMSProp keeps jittering.

On the **elongated bowl** (G-673) of the IIT feature (Figure 1), each optimizer gets a **learning rate** (G-1068) that works for it. Steps until the loss is within 0.01 of its minimum (Notebook):

Optimizer	η	Steps	Still moving at the end?
Gradient descent	0.3	61	no
Momentum, $\beta = 0.9$	0.1	64	no
AdaGrad	2	44	no
RMSProp, $\beta = 0.9$	0.3	48	yes: up to 0.15 from the best
Adam	0.5	42	no

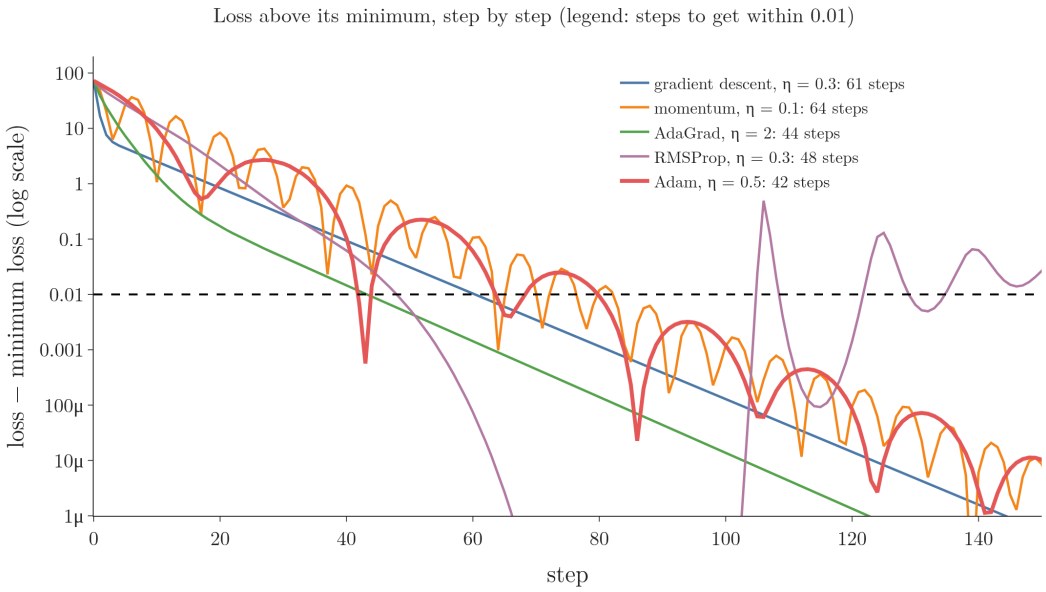


Figure 5: The loss above its minimum at every step for the five runs of the table, on a log scale. The dashed line is the 0.01 target; the legend gives the step at which each run first gets below it.

In Figure 5 the vertical axis is how far the loss is above its lowest possible value, on a log scale: each labelled line is 10 times the one below it (μ means one millionth, so 100μ is 0.0001), so the lower a curve, the closer the run is to the minimum. The dashed line is the 0.01 target.

Figure 5 shows the same runs step by step. The step counts in the table are the first time a run gets within 0.01; the curves show what happens next. Gradient descent and **AdaGrad** (G-168) keep falling smoothly. Momentum and Adam cross the line, swing back above it, and settle in smaller and smaller swings. RMSProp falls steadily, then very steeply after step 45, then jumps back up near step 105 and keeps bouncing around 0.01 to the end.

Momentum is no faster than gradient descent here (64 steps against 61). Its speed carries it past the minimum on the way down, so its loss rises and falls several times before it first gets within 0.01 (the bumps in its curve in Figure 5). These are the overshoot and oscillation of the table in section 3; on a small convex bowl that a plain step already crosses quickly, the swings cost the steps the speed gained.

Adam (G-169)'s path shows both behaviours. Adam moves in m and b together from the start, like AdaGrad and **RMSProp** (G-1697), instead of the "L" of gradient descent. Near the minimum it swings around a few times, each swing smaller, the momentum part, and then settles. RMSProp, with steps of about η even near the minimum, keeps jittering up to 0.15 away. On a convex bowl like this one (a **convex function**, G-476) the differences are small; Adam's strengths matter most on the complex, non-convex losses of real networks (**non-convex functions**, G-1333).

7. Adam on real data: MNIST

> **Key point:** On MNIST, Adam and RMSProp train fastest and are close to each other; both are far ahead of plain SGD and AdaGrad, and ahead of **momentum** (G-1258) and NAG.

The setup:

- **data:** the **MNIST** (G-1249) handwritten digits (the MNIST data), 10,000 training images, each with 784 pixel **features** (G-772; input variables) and the digit as **target** (G-1949; the output we predict), and the 10,000 test images for validation (checking accuracy on images the network did not train on);
- **network:** hidden layers of 128 and 64 **ReLU** (G-1668; each node outputs its input if positive, else 0) nodes and a softmax output (10 probabilities, one per digit);
- **training:** **batch size** (G-267) 64, 20 **epochs** (G-696), 3 seeds;
- **learning rates** (G-1068): 0.01 for SGD, momentum, **NAG** (G-1315) ($\beta = 0.9$) and **AdaGrad** (G-168); 0.001, Keras' default, for RMSProp and Adam.

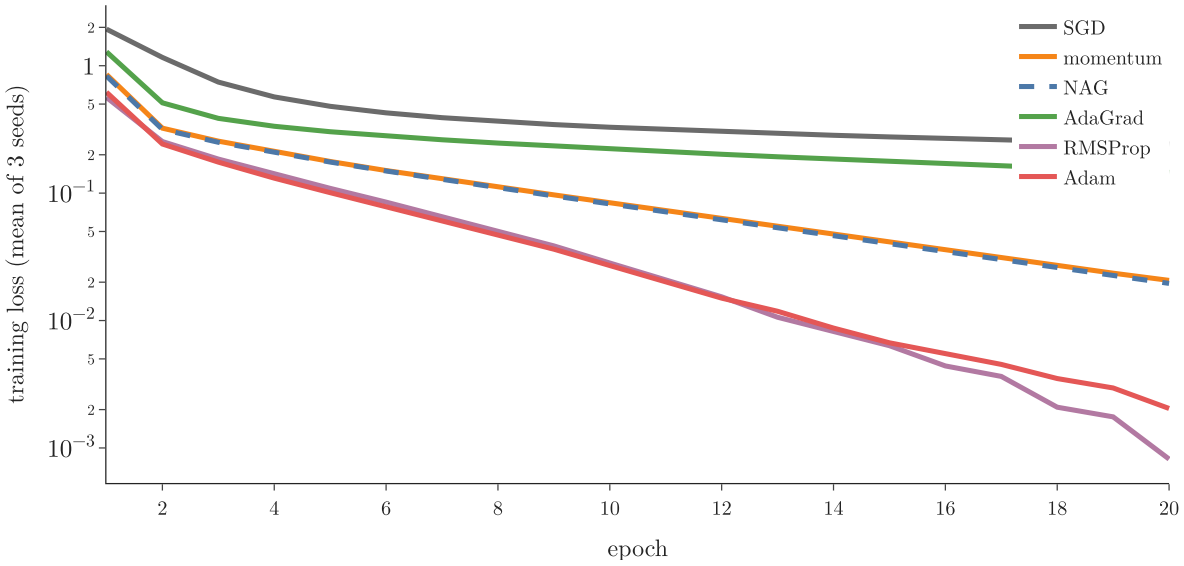


Figure 6: Training loss on MNIST per epoch for six optimizers, mean of 3 seeds

In Figure 6 each curve is the training loss after each epoch, on a log scale (each labelled

power of ten is 10 times the one below; the small numbers between mark 2, 3, ... times it). A lower curve is better.

Figure 6 and the Notebook give, as means over 3 seeds:

	SGD	Momentum	NAG	AdaGrad	RMSProp	Adam
Learning rate	0.01	0.01	0.01	0.01	0.001	0.001
Training loss, epoch 1	1.94	0.85	0.83	1.29	0.56	0.62
Training loss, epoch 5	0.48	0.18	0.18	0.30	0.11	0.10
Training loss, epoch 20	0.24	0.021	0.020	0.15	0.0008	0.0020
Validation accuracy, epoch 20	0.917	0.949	0.949	0.934	0.956	0.954

Adam behaves like both of its parents. Adam gets the speed of momentum and the per-parameter learning rates of RMSProp, so it is far ahead of plain SGD and AdaGrad, and ahead of momentum and NAG from the first epochs on. Against RMSProp alone the race is close on this small network: Adam is slightly lower by epoch 5, RMSProp slightly lower by epoch 20, and their validation accuracies differ by about 0.001 (0.9556 against 0.9542; Notebook). The learning rates are each **optimizer** (G-1401)'s usual value, not tuned, so small differences between the leaders should not be over-read; Adam's advantage is that it does well across many problems with these defaults (section 8).

8. Which optimizer to use

> **Key point:** Start with Adam. If the results are not good, try RMSProp, and sometimes **momentum** (G-1258). There is no single best optimizer: tune the choice like any hyperparameter.

There are more **optimizers** (G-1401) than the five covered here, such as AdaDelta and Nadam (Adam with Nesterov momentum). Which one to use has no clear answer: a large comparison across many tasks found the adaptive methods fairly robust, but no single best algorithm (Goodfellow et al. 2016, §8.5.4). A practical order:

1. **Adam** is a good starting point; over recent years it has given good results on many kinds of problems.
2. If **Adam** (G-169)'s results are not good, try **RMSProp**; sometimes **momentum** works best.

3. Treat the optimizer as a **hyperparameter** (G-910; a setting chosen before training; Bayesian search with Optuna covers tuning in general): try several and keep the one that does best on validation data, for example with **Keras Tuner** (G-1001; choosing the optimizer with Keras Tuner).

Adam is generally regarded as fairly robust to the choice of its hyperparameters, though the **learning rate** (G-1068) sometimes needs changing from the suggested default of 0.001 (Goodfellow et al. 2016, §8.5.3). Its authors describe the hyperparameters as having intuitive meanings and typically requiring little tuning (Kingma and Ba 2015).

> **Extra:** Ruder (2016, §4.10) recommends the adaptive methods when the data is sparse or the network deep or complex, and notes that Kingma and Ba found the bias correction helps Adam slightly outperform **RMSProp** (G-1697) towards the end of optimisation, as gradients become sparser.

9. Adam in Keras

> **Key point:** `keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)`, or simply `optimizer="adam"`.

> **Python: Adam** (G-169) in Keras.

```
import keras
opt = keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9,
                            beta_2=0.999)
model.compile(optimizer=opt, loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

The defaults match the paper's except `epsilon=1e-7` (paper: 10^{-8}) (Keras `Adam` documentation).

10. Summary

	Momentum	RMSProp	Adam
EWMA of gradients	yes (v_t)	no	yes (m_t, $\beta_1 = 0.9$)
EWMA of squared gradients	no	yes ($\beta = 0.9$)	yes (v_t, $\beta_2 = 0.999$)
Learning rate per parameter	no	yes	yes
Bias correction	no	no	yes, divide by $1 - \beta^t$
Default learning rate in Keras	0.01 (SGD)	0.001	0.001

- Adam = **momentum** (G-1258)'s average of gradients + RMSProp's average of squared gradients + a correction for starting at 0, because the two ideas are independent and, without the correction, the first steps would be badly scaled.
- The update divides the corrected average gradient by the root of the corrected average squared gradient (the formula of section 4), so it gets momentum's speed and a **learning rate** (G-1068) per parameter at once.
- Defaults: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ (β_2 nearer 1, because the squared-gradient average needs many gradients to be reliable); t counts the updates; **Adam** (G-169) is fairly robust to these, so usually only the learning rate needs changing.
- Adam is the usual first choice; RMSProp and momentum are the usual alternatives. No **optimizer** (G-1401) wins everywhere, so treat the optimizer as a hyperparameter and keep the one that does best on validation data.

11. Sources

Built from

- CampusX, "Adam Optimizer Explained in Detail with Animations | Optimizers in Deep Learning Part 5", YouTube, https://www.youtube.com/watch?v=N5AynalXD9g

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §8.5.3 Adam (algorithm 8.7), §8.5.4 Choosing the right optimization algorithm.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. ICLR 2015. arXiv:1412.6980. Algorithm 1, §2, §3 (a reliable second-moment estimate needs an average over many gradients); §6.4 (β_2 close to 1 is required for robustness to sparse gradients).
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747. §4.6 Adam, §4.10 Which optimizer to use?
- Keras API documentation: `Adam` optimizer, keras.io/api/optimizers/adam.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Adam (G-169)	An optimizer, the rule that updates a network's weights to reduce the loss: a variant of gradient descent that keeps running averages of past gradients and of their squares, giving each weight its own step size; fairly robust to its settings, so a common default.
First moment m_t (G-780)	A running average of the recent gradients in which older ones count less (an EWMA), so it estimates the gradient's mean; written m_t. Adam uses it in its update.
Second moment v_t (G-1759)	In Adam, the running average (EWMA) of the squared gradient, an estimate of its mean square; dividing the step by $\sqrt{v_t}$ gives each weight its own step size, as in RMSProp.
Bias correction (G-285)	A fix for a running average (EWMA) that starts at 0 and is too small in the first steps: divide it by $1 - \beta^t$.
β_1, β_2 (G-6)	Two numbers between 0 and 1 that set how much of the past Adam's two running averages keep (its decay factors): β_1 for the average of gradients m_t, β_2 for the average of squared gradients v_t. Defaults 0.9 and 0.999.
Optimizer (G-1401)	The rule that turns gradients into weight updates, such as plain gradient descent or Adam.
Convolutional neural network (CNN) (G-484)	A neural network that slides small filters over its input to find patterns such as edges, using at least one convolutional layer; the standard network for images.
Recurrent neural network (RNN) (G-1647)	A network whose hidden-layer output is fed back in, so it remembers earlier steps of a sequence.
Momentum (optimizer) (G-1258)	Gradient descent that keeps part of its past movement: each step follows an average of past gradients in which older ones count less and less (a velocity, an exponentially decaying average).

Term	Meaning
Nesterov accelerated gradient (NAG) (G-1315)	A version of momentum that first takes the momentum jump and then measures the gradient where it lands (the look-ahead point); seeing the slope ahead lets it brake before the minimum and swing less.
AdaGrad (G-168)	An optimizer that gives each parameter its own learning rate, $\eta/(\sqrt{v_t}+\epsilon)$, where v_t sums its past squared gradients.
RMSProp (G-1697)	An optimizer that divides each parameter's step by the root of a running average (EWMA) of its recent squared gradients, $v_t = \beta v_{t-1} + (1-\beta)g_t^2$; old gradients fade, so unlike AdaGrad the learning rate does not shrink to nothing.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Exponentially weighted moving average (EWMA) (G-735)	A running average in which recent values count most and older values count less and less, updated as $V_t = \beta V_{t-1} + (1-\beta)\theta_t$. Momentum and Adam use it to average the gradients.
Elongated bowl (G-673)	A loss surface stretched far more in one direction than another, with long elliptical contours, as a sparse feature produces. Gradient descent on it wastes steps moving along the steep direction and crawls along the flat one.
Convex function (G-476)	A function where a straight line between any two points of its curve never goes below the curve; it has a single minimum.
Non-convex function (G-1333)	A function whose curve is not one simple bowl: some straight line between two of its points (a chord) lies below part of the curve. It can have several local minima.
MNIST (G-1249)	A dataset of about 70,000 handwritten-digit images of 28×28 pixels.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.

Term	Meaning
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Batch size (G-267)	The number of rows in each batch; a hyperparameter.
Epoch (G-696)	One full update of the parameters using the whole training set.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Keras Tuner (G-1001)	A Python library (`keras_tuner`) that searches for good hyperparameters of a Keras model.